

SP BIOINSPIRADOS

SOLUCIÓN ÓPTIMA | A.GENÉTICO | COL.HORMIGAS| GREEDY

RESUMEN

El estudio compara diferentes métodos para resolver el Problema del Agente Viajero (TSP): un modelo exacto en GAMS, un algoritmo genético (GA), un algoritmo de colonia de hormigas (ACO) y un heurístico Greedy. Los resultados muestran que Greedy fue el más rápido y eficiente, con soluciones cercanas al óptimo y tiempos mínimos. El GA ofreció buen equilibrio entre precisión y tiempo, mientras que el ACO tuvo mayor capacidad de exploración pero más costo computacional. El modelo exacto garantizó la mejor solución, aunque con tiempos muy altos. En general, no existe un método único superior; la elección depende del balance entre tiempo, precisión y recursos disponibles.

MODELO MATEMÁTICO

obj.. $z = e = \sum_{(i,j)} x_{ij} c_{ij}$ MINIMIZA EL COSTO TOTAL DEL RECORRIDO ENTRE LOS NODOS.

$r1(i) \cdot \sum_j x_{ij} = e = 1$ ASEGURA QUE DE CADA NODO SALGA UNA SOLA RUTA.

$r2(j) \cdot \sum_i x_{ij} = e = 1$ ASEGURA QUE A CADA NODO LLEGUE UNA SOLA RUTA.

$r3(i,j) \text{ } \$ (ord(i) > 1 \wedge ord(j) > 1 \wedge ord(i) \neq ord(j)) \cdot$ EVITA SUBTOURS MEDIANTE LA RESTRICCIÓN MTZ (MANTIENE UN SOLO CIRCUITO).

$u_i - u_j + card(i) x_{ij} \leq card(i) - 1$

DESCRIPCIÓN DEL PROBLEMA

El TSP consiste en que un viajero debe recorrer n ciudades sin repetir ninguna y volver al inicio, buscando que la distancia total sea mínima. Cada par de ciudades tiene una distancia asociada, y el reto es encontrar el orden óptimo de visita. Para este proyecto se generaron 10 redes aleatorias de 40 nodos, y luego se probaron los tres métodos mencionados. La comparación permite ver qué técnica obtiene mejores resultados en menos tiempo, especialmente cuando el número de nodos aumenta.

PARAMETROS GA

- pop_size = 220 → Tamaño de la población.
- generations = 1200 → Número máximo iteraciones evolutivas.
- elite_size = 2 → Mejores individuos que pasan sin cambios.
- tournament_k = 3 → Tamaño del torneo.
- mutation_prob = 0.25 → Probabilidad de aplicar mutación.
- crossover_perc = 1.0 → Porcentaje de descendencia generada
- patience = 250 → Paro anticipado si no hay mejora
- Codificación: cromosoma = permutación de 0 a n-1
- Función fitness: 1/(1+L), donde L es la longitud del tour.

PARAMETROS ACO

$NUM_{ants} = 30$ $\beta = 0.5$

$NUM_{iteraciones} = 200$ $\rho = 0.5$

$\alpha = 1.0$ $q = 100.0$

RESULTADOS SOLO EN 40 NODOS

40 nodos								
Instancia	Exacto	Greedy	ACO	GA	Time Exacto	Time Greedy	Time ACO	Time GA
1	132.92	132.999	132.94	132.98	45.91	2.39	7.75	5.77
2	1.3249	1.402	1.39	1.43	2.901	4.35	6.53	7.78
3	0.9295	0.989	0.95	0.94	76.06	2.65	7.80	5.05
4	133.03	133.066	133.11	133.09	69.24	2.06	6.72	7.09
5	33.906	34.052	33.94	33.93	6.196	2.06	7.55	6.28
6	0.9712	1.047	0.99	0.97	14.64	2.08	6.82	5.28
7	33.992	34.085	34.05	34.01	67.09	2.09	7.25	6.83
8	1.028	1.106	1.05	1.09	38.67	2.96	7.76	4.63
9	1.3778	1.429	1.33157	1.33160	133.1	2.50	7.06	11.19
10	0.982	1.091	0.9839	0.9821	80.8	2.05	7.68	5.82
Tiempo promedio:					53.461	2.519	7.292	6.572

100 nodos								
Instancia	Exacto	Greedy	ACO	GA	Time Exacto	Time Greedy	Time ACO	Time GA
1	2.292	2.326	2.38	2.46	391.388	38.65	34.52	40.88
2	2.227	2.302	2.35	2.31	400.072	37.92	34.49	49.26
3	1.969	1.950	2.04	2.03	400.973	36.97	33.73	50.87
4	1.498	1.519	1.55	1.56	163.756	39.27	33.55	50.8
5	1.916	2.080	1.98	1.99	248.747	38.68	34.18	50.85
6	134.390	133.555	133.59	133.57	3.36	39.55	34.55	50.89
7	35.766	34.589	34.55	34.57	3.182	38.35	33.16	50.76
8	---	1.540	1.59	1.66		38.28	34.32	40.94
9	134.353	133.648	133.68643	133.55020	2.938	38.26	39.82	51.25
10	35.163	34.376	34.4957	34.6581	4.803	36.00	35.15	50.73
Tiempo promedio:					179.913	38.193	34.746	48.723

HEURISTICO

Fue el método más rápido en todas las pruebas, manteniendo tiempos bajos y estables. Aunque su precisión disminuye ligeramente con más nodos, sigue ofreciendo resultados bastante cercanos al óptimo, como se observa en varias celdas amarillas, lo que lo hace ideal para obtener soluciones rápidas y eficientes.

COLONIA DE HORMIGAS

Tuvo un desempeño intermedio, con resultados cercanos al óptimo en varias instancias y tiempos moderados. A medida que el problema crece, mantiene buena calidad de solución, pero el incremento de iteraciones y feromonas afecta su tiempo de ejecución.

ALGORITMO GENETICO

Fue el más consistente en aproximarse al método exacto, especialmente en las instancias más grandes. Sus soluciones resaltadas en amarillo confirman su capacidad de adaptación y exploración global. Aunque su tiempo es mayor que el Greedy, conserva una excelente relación entre precisión y rendimiento conforme aumenta el tamaño del problema.

OBSERVACIONES

De las 40 instancias evaluadas, 38 generaron soluciones factibles dentro del límite de 400 segundos, cumpliendo con la condición de visitar cada nodo una sola vez. Al ampliar el tiempo máximo de ejecución en el servidor NEOS, los tiempos aumentaron considerablemente, llegando a entre 30 minutos y 1 hora sin devolver resultados concretos. Esto evidencia las limitaciones del método exacto, que se vuelve ineficiente a partir de 100 nodos, incluso con recursos externos. Si bien NEOS permite obtener soluciones de referencia, su salida requiere preprocesamiento adicional y en algunos casos no entrega resultados tras largos periodos de espera. También se probó una versión mejorada del heurístico Greedy con 2-Opt, que ofreció mejores recorridos locales, pero con tiempos de ejecución superiores a una hora, por lo que fue descartada del análisis comparativo.

CONCLUSIONES

Los resultados muestran que no existe un único método ideal para todos los tamaños de problema, sino que el desempeño depende de la escala y los objetivos (precisión o rapidez). En las pruebas con 40 nodos, el Algoritmo Genético (GA) fue el que más se aproximó al método exacto, con un error promedio de 1.63 %, y la mayoría de sus valores marcados en amarillo, lo que refleja su capacidad de exploración y estabilidad al encontrar soluciones casi óptimas. En contraste, el Greedy mantuvo los menores tiempos de ejecución (2.52 s en promedio), aunque con un error ligeramente mayor (3.65 %), lo que lo hace ideal cuando se busca eficiencia sin exigir exactitud total. Al aumentar a 100 nodos, se evidenció la limitación del método exacto, con tiempos promedio superiores a 179 s y algunas instancias sin respuesta. En este escenario, tanto el GA como el Greedy siguieron destacando: los valores en amarillo se concentran principalmente en el GA, que conservó la mejor precisión (1.28 % de error), mientras que el Greedy mantuvo una excelente velocidad (38.19 s) con una desviación mínima (1.30 %). Además, el ACO también logró acercarse al óptimo en algunas instancias, mostrando un desempeño estable y competitivo, aunque con tiempos de ejecución algo mayores que los del GA y Greedy.

VENTAJAS

El Algoritmo Genético combina precisión y adaptabilidad, siendo ideal cuando se busca una solución muy cercana al óptimo en problemas complejos, ya que su estructura evolutiva le permite explorar eficientemente el espacio de soluciones. Por otro lado, el ACO también ofrece una buena aproximación al óptimo gracias a su mecanismo de aprendizaje colectivo, aunque con tiempos de cómputo mayores. El Greedy, en cambio, presenta una ventaja clara en simplicidad y rapidez, obteniendo resultados razonablemente buenos en una fracción del tiempo de los demás. Finalmente, el método Exacto garantiza la mejor solución posible, pero con un costo computacional elevado. En conjunto, los resultados evidencian que GA logra el mejor equilibrio entre exactitud y estabilidad, mientras que Greedy es la mejor opción cuando se prioriza la eficiencia temporal.

Método	Error promedio (40 nodos)	Error promedio (100 nodos)
Greedy	3.65%	1.30%
ACO	1.98%	1.43%
GA	1.63%	1.28%

[Link a Github con Documentación](#)

BIBLIOGRAFÍA

Alexander, A., & Sriwindono, H. (2020). The comparison of genetic algorithm and ant colony optimization in completing travelling salesman problem. In Proceedings of the 2nd International Conference of Science and Technology for the Internet of Things (ICSTI 2019) (pp. ...). EAI. <https://doi.org/10.4108/eai.20-9-2019.2292121> . eudl.eu

Boyko, N., & Pytel, A. (2020). Aspects of the study of genetic algorithms and mechanisms for their optimization for the travelling salesman problem. International Journal of Computing, 20(4), ... <https://doi.org/10.47839/ijc.20.4.2442>. computingonline.net

Chalarux, T., & Sripratak, P. (2020?). Worst case analyses of nearest neighbor heuristic for finding the minimum weight k-cycle. CURRENT Applied Science and Technology. <https://doi.org/%E2%80%A6> . li01.tci-thaijo.org

Diaby, M. (2008). A $O(n^8) \times O(n^7)$ linear programming model of the traveling salesman problem. arXiv. <https://arxiv.org/abs/0803.4354>. arXiv

Eido, W.M., & Ibrahim, I.M. (2025). Ant Colony Optimization (ACO) for Traveling Salesman Problem: A Review. Asian Journal of Research in Computer Science, 18(2), 20-45. <https://doi.org/10.9734/ajrcos/2025/v18i2559> . journalajrcos.com

Islam, A.H.M., Tanzim, M., Afreen, S., & Rozario, G. (2019). Evaluation of ant colony optimization algorithm compared to genetic algorithm, dynamic programming and branch and bound algorithm regarding travelling salesman problem. Global Journal of Computer Science and Technology, 19(D3), 7-12. Retrieved from <https://computerresearch.org/index.php/computer/article/view/1842> . computerresearch.org

Kumar, S., & Munapo, E. (2021?). A greedy reconstruction heuristic for solving the minimum travelling salesman tour problem. Journal of Graphic Era University. <https://doi.org/10.13052/jgeu0975-1416.1321>. journal.riverpublishers.com

Larrañaga, P., Kuijpers, C.M.H., Murga, R.H., Inza, I., & Dizdarevic, S. (1999). Genetic algorithms for the travelling salesman problem: A review of representations and operators. Artificial Intelligence Review, 13(2), 129-170. <https://doi.org/10.1023/A:1006529012972> . Tilburg University Research Portal

Orman, A.J., & Williams, H.P. (n.d.). A survey of different integer-programming formulations of the travelling salesman problem. Retrieved from https://www.dei.unipd.it/~fisch/ricop/OR2/Survey_compact_TSP_models.pdf . dei.unipd.it

Rahman, M.Z., Sheikh, S.R., Islam, A., & AzizurRahman, M. (2024). Improvement of the nearest neighbor heuristic search algorithm for traveling salesman problem. Journal of Engineering Advancements, ... <https://doi.org/10.38032/jea.2024.01.004>. scienpg.com

Shyamala, K., & Sudha Prabha, S. (2014). An ant colony optimization approach to solve travelling salesman problem. International Journal on Recent and Innovation Trends in Computing and Communication, 2(12), 3966-3971. https://www.academia.edu/17700194/An_Ant_Colony_Optimization_approach_to_solve_Travelling_Salesman_Problem. ijritcc.org

Zanaj, B., & Zanaj, E. (2016). Review of traveling salesman problem for the genetic algorithms. Journal of Information Sciences and Computing Technologies, 5(3), 534-545. Retrieved from <http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/> . scitecresearch.com
